

## Implementing Fabric Channels for Privacy

To create and configure multiple channels in a Hyperledger Fabric network, deploy chaincode on a specific channel, and demonstrate how Fabric channels provide data privacy and isolation between organizations.

```
cd ~/fabric-dev/fabric-samples/test-network
```

```
./network.sh down
```

```
./network.sh up createChannel -ca
```

```

$ ./configtxlator proto_encode --input /home/ram/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --output /home/ram/fabric-dev/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-09-08 17:58:52.394 IST 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
2026-09-08 17:58:52.403 IST 0002 INFO [channelCmd] update → Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined

```

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/
```

```
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051

export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

#### 4. Check **mychannel**

None

```
peer channel list
```

```
(fabric@dev) [~/fabric-dev/fabric-samples/test-network]$ peer channel list
2026-09-08 18:07:37.846 IST 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
Channels peers has joined:
mychannel
privatechannel
```

#### 5. Create **privatechannel**

None

```
./network.sh createChannel -c privatechannel
```

```

{"policies":{"values":{"version":"0"},""},"version":"0"},"}}}
++ configtxlator proto encode --input /home/ram/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --output /home/ram/fabric-dev/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-09-08 18:01:14.025 IST 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
2026-09-08 18:01:14.034 IST 0002 INFO [channelCmd] update → Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'privatechannel'
Channel 'privatechannel' joined

```

## 6. Verify both channels

None

peer channel list

```

(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer channel list
2026-09-08 18:01:40.851 IST 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
Channels peers has joined:
mychannel
privatechannel

```

## 7. Deploy chaincode ONLY on privatechannel

This is the important part of the experiment.

None

```

./network.sh deployCC \
-c privatechannel \
-ccn basic \
-cc1 javascript \
-ccp ../asset-transfer-basic/chaincode-javascript

```

```

+ peer lifecycle chaincode querycommitted --channelID privatechannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'privatechannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'privatechannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'privatechannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID privatechannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'privatechannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'privatechannel'
Chaincode initialization is not required

```

## 8. Initialize the private-channel ledger

None

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile "$ORDERER_CA" \  
-C privatechannel \  
-n basic \  
-c '{"function":"InitLedger","Args":[]}' \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
"${PWD}/organizations/peerOrganizations/org2.example.com/tlsca  
/tlsca.org2.example.com-cert.pem" \  
--waitForEvent
```

```
(ram@ram) [~/fabric-dev/fabric-samples/test-network]  
$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile "$ORDERER_CA" \  
-C privatechannel \  
-n basic \  
-c '{"function":"InitLedger","Args":[]}' \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem" \  
--waitForEvent  
2026-09-08 18:03:13.283 IST 0001 INFO [chaincodeCmd] ClientWait → txid [e64d784fdbdc29c69e809d427c274fddb66c38efb72314533f76fb4d63abf6f2] committed with  
status (VALID) at localhost:7051  
2026-09-08 18:03:13.291 IST 0002 INFO [chaincodeCmd] ClientWait → txid [e64d784fdbdc29c69e809d427c274fddb66c38efb72314533f76fb4d63abf6f2] committed with  
status (VALID) at localhost:9051  
2026-09-08 18:03:13.291 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200
```

This is the same `InitLedger` operation specified by the manual, with the ad  
for the endorsement setup you've already encountered.  [View on GitHub](#)  
You should get a successful transaction.

9. Query assets from `privatechannel`

## 9. Query assets from `privatechannel`

None

```
peer chaincode query \  
-C privatechannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

The output should look like this:

None

```
[
  {
    "AppraisedValue": 300,
    "Color": "blue",
    "ID": "asset1",
    "Owner": "Tomoko",
    "Size": 5,
    "docType": "asset"
  },
  {
    "AppraisedValue": 400,
    "Color": "red",
    "ID": "asset2",
    "Owner": "Brad",
    "Size": 5,
    "docType": "asset"
  }
]
```

```
(ram@ram) ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-C privatechannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

## 10. Demonstrate channel isolation

Now query **asset1** from **mychannel**:

None

```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
```

## Important

Do **not** expect this to return the asset from **privatechannel**.

```
(ram@ram):~/fabric-dev/fabric-samples/test-network$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
Error: endorsement failure during query. response: status:500 message:"make sure the chaincode basic has been successfully defined on channel mychannel and try again: chaincode basic not found"
```

Important  
Do not expect this to return the asset from privatechannel.

## 11. Optional — prove privatechannel has the asset

Run:

```
None
peer chaincode query \
-C privatechannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
```

You should get:

```
None
{
  "AppraisedValue": 300,
  "Color": "blue",
  "ID": "asset1",
  "Owner": "Tomoko",
  "Size": 5,
  "docType": "asset"
}
```

Then compare:

```
None
privatechannel → asset1 EXISTS
```

mychannel1 → asset1 from privatechannel DOES NOT EXIST

That's the core demonstration of channel isolation.

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
-C privatechannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}
```

12. Stop the network